What are Servlets?

Java Servlets are programs that run on a Web or Application server and act as a middle layer between a requests coming from a Web browser or other HTTP client and databases or applications on the HTTP server.

Using Servlets, you can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically.

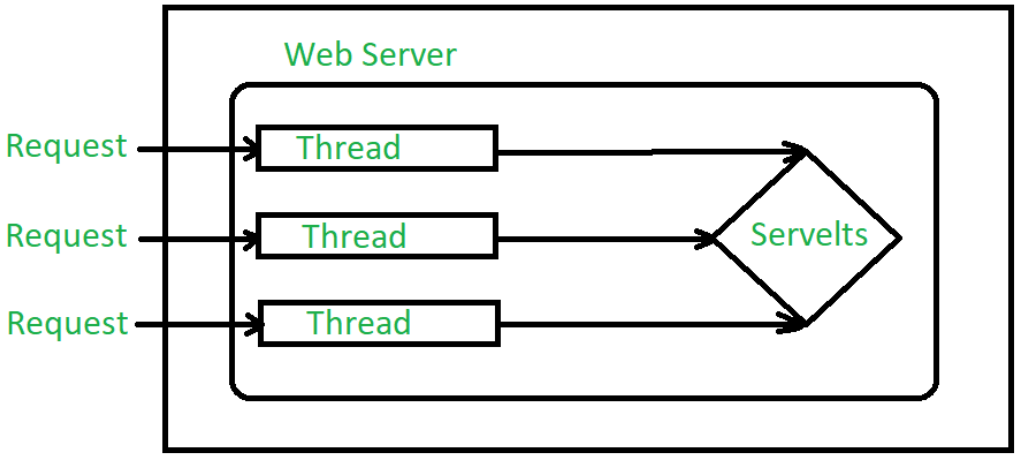
Java Servlets often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But Servlets offer several advantages in comparison with the CGI.

- Performance is significantly better.
- Servlets execute within the address space of a Web server. It is not necessary to create a separate process to handle each client request.
- Servlets are platform-independent because they are written in Java.
- Java security manager on the server enforces a set of restrictions to protect the resources on a server machine. So servlets are trusted.
- The full functionality of the Java class libraries is available to a servlet. It can communicate with applets, databases, or other software via the sockets and RMI mechanisms that you have seen already.

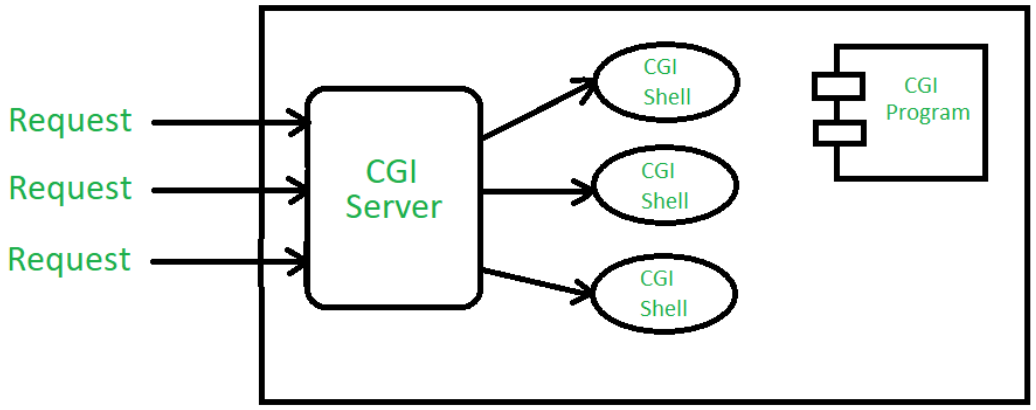
Difference between Java Servlet and CGI

The world has changed into a mobile-first era but even today, none of the applications could emerge as effective as the web-based apps. Surfacing on top of this is the prevalence of progressive web apps that perform functions identical to mobile apps. In this article, we will understand the difference between the two functionalities in web-based applications namely servlets and CGI.

Servlet is a Java class that is used to extend the capabilities of servers that host applications accessed by means of a request-response model. Servlets are mainly used to extend the applications hosted by web servers, however, they can respond to other types of requests too. For such applications, HTTP-specific servlet classes are defined by Java Servlet technology. All the programs of Servlets are written in JAVA and they get to run on JAVA Virtual Machine. The following image describes how a request from clients is served with the help of threads:



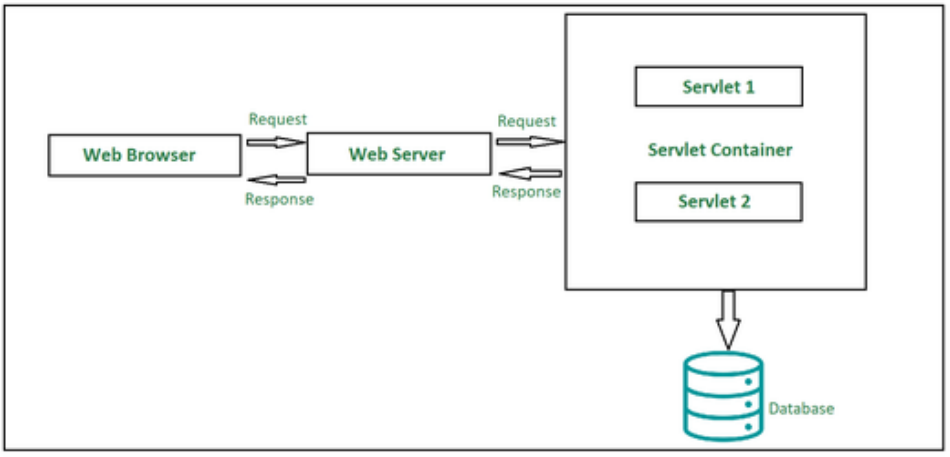
Common Gateway Interface(CGI): The Common Gateway Interface (CGI) provides the middleware between WWW servers and external databases and information sources. The World Wide Web Consortium (W3C) defined the Common Gateway Interface (CGI) and also defined how a program interacts with a HyperText Transfer Protocol (HTTP) server. The Web server typically passes the form information to a small application program that processes the data and may send back a confirmation message. This process or convention for passing data back and forth between the server and the application is called the common gateway interface (CGI). The following image describes how a web server acts as an intermediate between the CGI program and the client browser.



Basis	Servlet	CGI
Approach	It is thread based i.e. for every new request new thread is created.	It is process-based i.e. for every new request new process is created.

Basis	Servlet	CGI
Language Used	The codes are written in JAVA programming language.	The codes are written any programming language.
Object-Oriented	Since codes are written in Java, it is object oriented and the user will get the benefits of OOPs	Since codes are written in any language, all the languages are not object-oriented thread-based. So, the user will not get the benefits of OOPs
Portability	It is portable.	It is not portable.
Persistence	It remains in the memory until it is not explicitly destroyed.	It is removed from the memory after the completion of the process-basedrequest.
Server Independent	It can use any of the web-server.	It can use the web-server that supports it.
Data Sharing	Data sharing is possible.	Data sharing is not possible.
Link	It links directly to the server.	It does not link the web server directly to the server.
HTTP server	It can read and set HTTP servers.	It can neither read nor set HTTP servers.
Cost	Construction and destruction of new threads are not costly.	Construction and destruction of the new processes are costly.
Speed	Its can speed is slower.	It can speed is faster.
Platform dependency	It can be Platform Independent	It can be Platform dependent.

Servlets Architecture



Servlet Architecture

1. Client
The client shown in the architecture above is primarily working as a medium who is sending out HTTP requests over to the web server and again processing the response it gets back from the server. As we can see in the diagram, our client here is the web browser.

2. Web Server
Primary job of a web server is to process the requests and responses that a user sends over time and maintain how a web user would be able to access the files that has been hosted over the server. The server we are talking about here is a software which manages access to a centralized resource or service in a network.. There are precisely two types of webservers:
1. Static web server
2. Dynamic web server

3. Web Container
Web container is another typical component in servlet architecture which is responsible for communicating with the servlets. Two prime tasks of a web container are:

- Managing the servlet lifecycle
- URL mapping

Web container sits at the server-side managing and handling all the requests that are coming in either from the servlets or from some JSP pages or potentially any other file system.

How does a Servlet Request flow?
Every servlet should override the following 3 methods namely:

1. init()
2. service()
3. destroy()

These methods are used to process the request from the user.

Following are the steps in which a request flows through a servlet which can be observed in the architecture diagram:

- The client sends over a request.
- The request is accepted by the web server and forwarded to the web container.
- In order to obtain the servlet's address, the web container traces *web.xml* file corresponding to the request *URL pattern.*
- By the time above process takes place, the servlet should have been instantiated and initialized. If the servlet has not been instantiated and initialized, *init()* method is invoked to serve the purpose.
- By passing *ServletRequest* and *Response object*, *public service()* method is called by the container.
- In the next step, the *ServletRequest* and *ServletResponse* objects are type-casted to *HttpServletRequest* and *HttpServletResponse* objects by the *public service()* method.
- Now *protected service()* method is called by the *public service()* method.
- The *protected service()* method dispatches the request to the correct handler method based on the type of request.
- When servlet container shuts down, it unloads all the servlets and calls *destroy()* method for each initialized servlets.

Advantages

- Prime functionality of a servlet is that they are independent of server configuration and they are pretty much compatible with any of the web servers
- Servlets are also protocol-independent supporting *FTP*, *HTTP*, *SMTP*, etc. protocols to the fullest.
- Until destroyed manually, servlets can be retained in the memory helping process several requests over time. Also, once a database connection is established, it can facilitate process several requests for a database in the very same database session.
- Servlets inherit Java's property of portability and hence are compatible with nearly any web server.
- Servlets are first converted into byte codes and then executed, which helps in increasing the processing time.

Disadvantages

- Designing a servlet can be pretty laborious.
- Exceptions need to be handled while designing a servlet since they are not thread-safe.
- Developers may need additional skills to program a servlet.

`@WebServlet`

The `@WebServlet` annotation is a powerful feature in Java Servlets that streamlines the development process by allowing developers to manage servlet configurations directly in the code. This enhances both productivity and maintainability in web application development.

Purpose of `@WebServlet`

1. **Servlet Declaration**: It allows developers to declare a servlet without the need for web.xml configuration. This is part of Java's annotation-based programming, introduced in Servlet 3.0.

2. **URL Mapping**: The `@WebServlet` annotation specifies the URL patterns that the servlet responds to. This means you can define which HTTP requests the servlet will handle directly in the servlet code.

Syntax

Here's a basic example of how to use the `@WebServlet` annotation:

```
@WebServlet("/hello") // Maps this servlet to the URL pattern /hello
public class HelloServlet extends HttpServlet {
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
    response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>Hello, World!</h1>");
        }
}
```

Advantages :

1. **Simplifies Configuration**: Eliminates the need to modify the `web.xml` file, making the servlet easier to read and maintain.
2. **Enhanced Readability**: Annotations make it clear what the servlet is doing and how it is configured right next to the servlet code.
3. **Flexibility**: You can easily change the URL mapping or parameters by just modifying the annotation without touching the XML configuration.
4. **Cleaner Code**: Reduces boilerplate code, keeping the servlet implementation focused on its logic rather than configuration.

Servlets API

Servlets are the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the webserver, process the request, produce the response, then send a response back to the webserver. In Java, to create web applications we use Servlets. To create Java Servlets, we need to use Servlet API which contains all the necessary interfaces and classes. Servlet API has 2 packages namely,

1. javax.servlet
2. javax.servlet.http

1. javax.servlet

This package provides the number of interfaces and classes to support Generic servlet which is protocol independent.

These interfaces and classes describe and define the contracts between a servlet class and the runtime environment provided by a servlet container.

Classes available in javax.servlet package:

Class Name	Description
GenericServlet	To define a generic and protocol-independent servlet.
ServletContextAttributeEvent	To generate notifications about changes to the attributes of the servlet context of a web application.
ServletContextEvent	To generate notifications about changes to the servlet context of a web application.
ServletInputStream	This class provides an input stream to read binary data from a client request.
ServletOutputStream	This class provides an output stream for sending binary data to the client.
ServletRequestAttributeEvent	To generate notifications about changes to the attributes of the servlet request in an application.
ServletRequestEvent	To indicate lifecycle events for a ServletRequest.
ServletRequestWrapper	This class provides the implementation of the ServletRequest interface that can be subclassed by developers to adapt the request to a Servlet.
ServletResponseWrapper	This class provides the implementation of the ServletResponse interface that can be subclassed by developers to adapt the response from a Servlet.

Interfaces available in javax.servlet package:

Interface Name	Description
Filter	To perform filtering tasks on either the request to a resource, or on the response from a resource, or both.
FilterChain	To provide a view into the invocation chain of a filtered request for a resource to the developer by the servlet container.
FilterConfig	To pass information to a filter during initialization used by a servlet container.
RequestDispatcher	It defines an object to dispatch the request and response to any other resource, means it receives requests from the client and sends them to a servlet/HTML file/JSP file on the server.
Servlet	This is the main interface that defines the methods in which all the servlets must implement. To implement this interface, write a generic servlet that extends javax.servlet.GenericServlet or an HTTP servlet that extends javax.servlet.http.HttpServlet.
ServletConfig	It defines an object created by a servlet container at the time of servlet instantiation and to pass information to the servlet during initialization.
ServletContext	It defines a set of methods that a servlet uses to communicate with its servlet container. The information related to the web application available in web.xml file is stored in ServletContext object created by container.
ServletContextAttributeListener	The classes that implement this interface receive notifications of changes to the attribute list on the servlet context of a web application.
ServletContextListener	The classes that implement this interface receive notifications about changes to the servlet context of the web application they are part of.

Interface Name	Description
ServletRequest	It defines an object that is created by servlet container to pass client request information to a servlet.
ServletRequestAttributeListener	To generate the notifications of request attribute changes while the request is within the scope of the web application in which the listener is registered.
ServletRequestListener	To generate the notifications of requests coming in and out of scope in a web component.
ServletResponse	It defines an object created by servlet container to assist a servlet in sending a response to the client.

Exceptions in javax.servlet package:

Exception Name	Description
ServletException	A general exception thrown by a servlet when it encounters difficulty.
UnavailableException	Thrown by a servlet or filter to indicate that it is permanently or temporarily unavailable.

2. javax.servlet.http

This package provides the number of interfaces and classes to support HTTP servlet which is HTTP protocol dependent.

These interfaces and classes describe and define the contracts between a servlet class running under HTTP protocol and the runtime environment provided by a servlet container.

Classes available in javax.servlet.http package:

Class Name	Description
Cookie	Creates a cookie object. It is a small amount of information sent by a servlet to a Web browser, saved by the browser, and later sent back to the server used for session management.
HttpServlet	Provides an abstract class that defines methods to create an HTTP suitable servlet for a web application.
HttpServletRequestWrapper	This class provides implementation of the HttpServletRequest interface that can be subclassed to adapt the request to a Servlet.
HttpServletResponseWrapper	This class provides implementation of the HttpServletResponse interface that can be subclassed to adapt the response from a Servlet.
HttpSessionBindingEvent	This events are either sent to an object that implements HttpSessionBindingListener when it is bound or unbound from a session, or to a HttpSessionAttributeListener that has been configured in the deployment descriptor when any attribute is bound, unbound or replaced in a session.
HttpSessionEvent	To represent event notifications for changes to sessions within a web application.

Interfaces available in javax.servlet.http package:

Interface Name	Description
HttpServletRequest	To provide client HTTP request information for servlets. It extends the ServletRequest interface.
HttpServletResponse	To provide HTTP-specific functionality in sending a response to client. It extends the ServletResponse interface.

Interface Name	Description
HttpSession	It provides a way to identify a user across web application/web site pages and to store information about that user.
HttpSessionActivationListener	Container to notify all the objects that are bound to a session that sessions will be passivated and that session will be activated.
HttpSessionAttributeListener	To get notifications of changes to the attribute lists of sessions within this web application, this listener interface can be implemented.
HttpSessionBindingListener	It causes an object to be notified by an HttpSessionBindingEvent object, when it is bound to or unbound from a session.
HttpSessionListener	To receive notification events related to the changes to the list of active sessions in a web application.

Servlet Interface

Servlet interface provides common behavior to all the servlets.Servlet interface defines methods that all servlets must implement.

Servlet interface needs to be implemented for creating any servlet (either directly or indirectly). It provides 3 life cycle methods that are used to initialize the servlet, to service the requests, and to destroy the servlet and 2 non-life cycle methods.

Methods of Servlet interface

There are 5 methods in the Servlet interface. The init, service and destroy are the life cycle methods of servlet. These are invoked by the web container.

Method	Description
public void init(ServletConfig config)	initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.
public void service(ServletRequest request,ServletResponse response)	provides response for the incoming request. It is invoked at each request by the web container.
public void destroy()	is invoked only once and indicates that servlet is being destroyed.

public ServletConfig getServletConfig()	returns the object of ServletConfig.
public String getServletInfo()	returns information about servlet such as writer, copyright, version etc.

SERVLET Classes Type :

1. GenericServlet class

GenericServlet class implements **Servlet**, **ServletConfig** and **Serializable** interfaces. It provides the implementation of all the methods of these interfaces except the service method.

GenericServlet class can handle any type of request so it is protocol-independent.

You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

Methods of GenericServlet class

There are many methods in GenericServlet class. They are as follows:

1. **public void init(ServletConfig config)** is used to initialize the servlet.
2. **public abstract void service(ServletRequest request, ServletResponse response)** provides service for the incoming request. It is invoked at each time when a user requests for a servlet.
3. **public void destroy()** is invoked only once throughout the life cycle and indicates that servlet is being destroyed.
4. **public ServletConfig getServletConfig()** returns the object of ServletConfig.
5. **public String getServletInfo()** returns information about servlet such as writer, copyright, version etc.
6. **public void init()** it is a convenient method for the servlet programmers, now there is no need to call super.init(config)
7. **public ServletContext getServletContext()** returns the object of ServletContext.
8. **public String getInitParameter(String name)** returns the parameter value for the given parameter name.
9. **public Enumeration getInitParameterNames()** returns all the parameters defined in the web.xml file.
10. **public String getServletName()** returns the name of the servlet object.
11. **public void log(String msg)** writes the given message in the servlet log file.
12. **public void log(String msg,Throwable t)** writes the explanatory message in the servlet log file and a stack trace.

2.HttpServlet class

The HttpServlet class extends the GenericServlet class and implements Serializable interface.
It provides http specific methods such as doGet, doPost, doHead, doTrace etc.

Methods of HttpServlet class

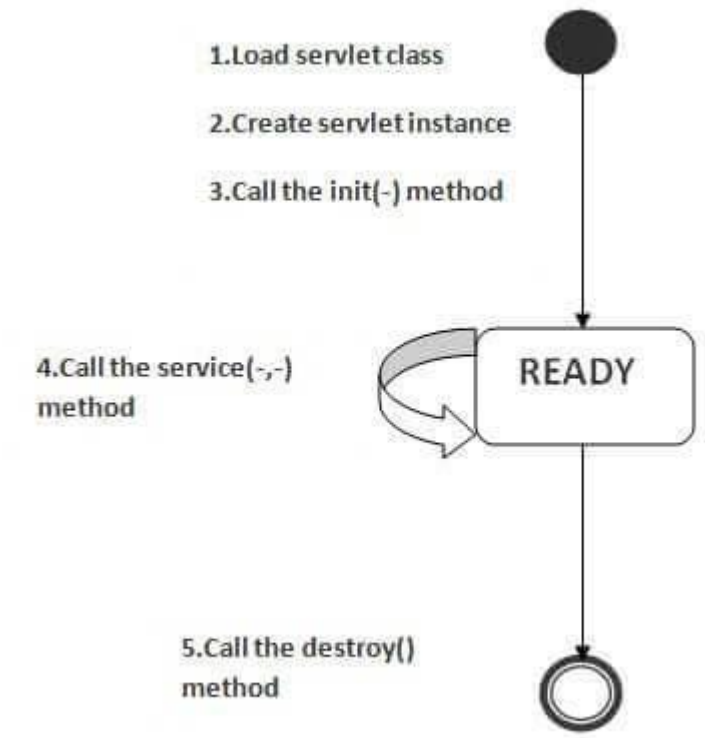
There are many methods in HttpServlet class. They are as follows:

1. **public void service(ServletRequest req,ServletResponse res)** dispatches the request to the protected service method by converting the request and response object into http type.
2. **protected void service(HttpServletRequest req, HttpServletResponse res)** receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.
3. **protected void doGet(HttpServletRequest req, HttpServletResponse res)** handles the GET request. It is invoked by the web container.
4. **protected void doPost(HttpServletRequest req, HttpServletResponse res)** handles the POST request. It is invoked by the web container.
5. **protected void doHead(HttpServletRequest req, HttpServletResponse res)** handles the HEAD request. It is invoked by the web container.
6. **protected void doOptions(HttpServletRequest req, HttpServletResponse res)** handles the OPTIONS request. It is invoked by the web container.
7. **protected void doPut(HttpServletRequest req, HttpServletResponse res)** handles the PUT request. It is invoked by the web container.
8. **protected void doTrace(HttpServletRequest req, HttpServletResponse res)** handles the TRACE request. It is invoked by the web container.
9. **protected void doDelete(HttpServletRequest req, HttpServletResponse res)** handles the DELETE request. It is invoked by the web container.
10. **protected long getLastModified(HttpServletRequest req)** returns the time when HttpServletRequest was last modified since midnight January 1, 1970 GMT.

Life Cycle of a Servlet (Servlet Life Cycle)

The web container maintains the life cycle of a servlet instance. Let's see the life cycle of the servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



A servlet life cycle can be defined as the entire process from its creation till the destruction. The following are the paths followed by a servlet.

- The servlet is initialized by calling the **init()** method.
- The servlet calls **service()** method to process a client's request.
- The servlet is terminated by calling the **destroy()** method.
- Finally, servlet is garbage collected by the garbage collector of the JVM.

The init() Method

The init method is called only once. It is called only when the servlet is created, and not called for any user requests afterwards. So, it is used for one-time initializations.

When a user invokes a servlet, a single instance of each servlet gets created, with each user request resulting in a new thread that is handed off to doGet or doPost as appropriate. The init() method simply creates or loads some data that will be used throughout the life of the servlet.

The init method definition looks like this −

```
public void init() throws ServletException {
```

```
  // Initialization code...
}
```

The service() Method

The service() method is the main method to perform the actual task. The servlet container (i.e. web server) calls the service() method to handle requests coming from the client(browsers) and to write the formatted response back to the client.

Each time the server receives a request for a servlet, the server spawns a new thread and calls service. The service() method checks the HTTP request type (GET, POST, PUT, DELETE, etc.) and calls doGet, doPost, doPut, doDelete, etc. methods as appropriate.

Here is the signature of this method −

```
public void service(ServletRequest request, ServletResponse response)
  throws ServletException, IOException {
}
```

The service () method is called by the container and service method invokes doGet, doPost, doPut, doDelete, etc. methods as appropriate. So you have nothing to do with service() method but you override either doGet() or doPost() depending on what type of request you receive from the client.

The doGet() and doPost() are most frequently used methods with in each service request. Here is the signature of these two methods.

The doGet() Method

A GET request results from a normal request for a URL or from an HTML form that has no METHOD specified and it should be handled by doGet() method.

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
  throws ServletException, IOException {
  // Servlet code
}
```

The doPost() Method

A POST request results from an HTML form that specifically lists POST as the METHOD and it should be handled by doPost() method.

```
public void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
  throws ServletException, IOException {
  // Servlet code
}
```

The destroy() Method

The destroy() method is called only once at the end of the life cycle of a servlet. This method gives your servlet a chance to close database connections, halt background threads, write cookie lists or hit counts to disk, and perform other such cleanup activities.

After the destroy() method is called, the servlet object is marked for garbage collection. The destroy method definition looks like this −

```
public void destroy() {
  // Finalization code...
}
```

Steps to create a servlet example

There are given 6 steps to create a **servlet example**. These steps are required for all the servers.

The servlet example can be created by three ways:

1. By implementing Servlet interface,
2. By inheriting GenericServlet class, (or)
3. By inheriting HttpServlet class

The mostly used approach is by extending HttpServlet because it provides http request specific method such as doGet(), doPost(), doHead() etc.

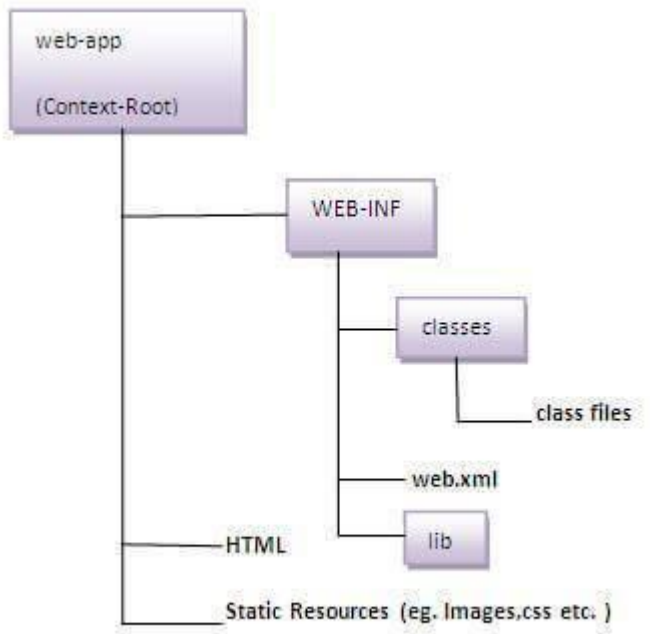
1. Create a directory structure
2. Create a Servlet
3. Compile the Servlet
4. Create a deployment descriptor
5. Start the server and deploy the project

6. Access the servlet

1)Create a directory structures

The **directory structure** defines that where to put the different types of files so that web container may get the information and respond to the client.

The Sun Microsystem defines a unique standard to be followed by all the server vendors. Let's see the directory structure that must be followed to create the servlet.



As you can see that the servlet class file must be in the classes folder. The web.xml file must be under the WEB-INF folder.

---

2)Create a Servlet

There are three ways to create the servlet.

1. By implementing the Servlet interface
2. By inheriting the GenericServlet class
3. By inheriting the HttpServlet class

The HttpServlet class is widely used to create the servlet because it provides methods to handle http requests such as doGet(), doPost, doHead() etc.

In this example we are going to create a servlet that extends the HttpServlet class. In this example, we are inheriting the HttpServlet class and providing the implementation of the doGet() method. Notice that get request is the default request.

Following is the sample source code structure of a servlet example to show Hello World −

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {

  private String message;

  public void init() throws ServletException {
    // Do required initialization
    message = "Hello World";
  }

  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
```

```
    // Set response content type
    response.setContentType("text/html");

    // Actual logic goes here.
    PrintWriter out = response.getWriter();
    out.println("<h1>" + message + "</h1>");
  }

  public void destroy() {
    // do nothing.
  }
}
```

DemoServlet.java

```
import javax.servlet.http.*;

import javax.servlet.*;

import java.io.*;

public class DemoServlet extends HttpServlet{

public void doGet(HttpServletRequest req,HttpServletResponse res)

throws ServletException,IOException

{

res.setContentType("text/html");//setting the content type

PrintWriter pw=res.getWriter();//get the stream to write the data

  //writing html in the stream

pw.println("<html><body>");
```

```
pw.println("Welcome to servlet");

pw.println("</body></html>");

  pw.close();//closing the stream

}}
```

Servlets - Form Data

GET Method

The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the **?** (question mark) symbol as follows −

```
http://www.test.com/hello?key1 = value1&key2 = value2
```

The GET method is the default method to pass information from browser to web server and it produces a long string that appears in your browser's Location:box. Never use the GET method if you have password or other sensitive information to pass to the server. The GET method has size limitation: only 1024 characters can be used in a request string.

This information is passed using QUERY_STRING header and will be accessible through QUERY_STRING environment variable and Servlet handles this type of requests using **doGet()** method.

POST Method

A generally more reliable method of passing information to a backend program is the POST method. This packages the information in exactly the same way as GET method, but instead of sending it as a text string after a ? (question mark) in the URL it sends it as a separate message. This message comes to the backend program in the form of the standard input

which you can parse and use for your processing. Servlet handles this type of requests using **doPost()** method.

Reading Form Data using Servlet

Servlets handles form data parsing automatically using the following methods depending on the situation −

1. **getParameter()** − You call request.getParameter() method to get the value of a form parameter.
2. **getParameterValues()** − Call this method if the parameter appears more than once and returns multiple values, for example checkbox.

3. **getParameterNames()** − Call this method if you want a complete list of all parameters in the current request.

Example with SessionContext :

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>ServletContext Demo</title>
</head>
<body>
   <h1>Welcome to ServletContext Demo</h1>
   <p><a href="servlet1">Visit DemoServlet1</a></p>
</body>
</html>
```

DemoServlet1.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DemoServlet1 extends HttpServlet {
```

```
    public void doGet(HttpServletRequest req, HttpServletResponse res) {
        try {
            res.setContentType("text/html");
            PrintWriter out = res.getWriter();

            // Set a ServletContext attribute
            ServletContext context = getServletContext();
            context.setAttribute("company", "IBM");

            out.println("<h1>Welcome to DemoServlet1</h1>");
            out.println("<p>Company attribute set to: " + context.getAttribute("company") +
"</p>");
            out.println("<p><a href='servlet2'>Visit DemoServlet2</a></p>");
            out.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

DemoServlet2.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DemoServlet2 extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) {
        try {
            res.setContentType("text/html");
            PrintWriter out = res.getWriter();

            // Retrieve the ServletContext attribute
            ServletContext context = getServletContext();
            String companyName = (String) context.getAttribute("company");

            out.println("<h1>Welcome to DemoServlet2</h1>");
            out.println("<p>Company attribute retrieved: " + companyName + "</p>");
            out.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

GET Method Example Using Form

HelloForm.java

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloForm extends HttpServlet {

  // Method to handle GET method request.
  public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Set response content type
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "Using GET Method to Read Form Data";
    String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " +
      "transitional//en\">\n";

    out.println(docType +
      "<html>\n" +
        "<head><title>" + title + "</title></head>\n" +
        "<body bgcolor = \"#f0f0f0\">\n" +
          "<h1 align = \"center\">" + title + "</h1>\n" +
          "<ul>\n" +
            "  <li><b>First Name</b>: "
            + request.getParameter("first_name") + "\n" +
            "  <li><b>Last Name</b>: "
            + request.getParameter("last_name") + "\n" +
```

```
            "</ul>\n" +
          "</body>"
        "</html>"
      );
  }

  // Method to handle POST method request.
  public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

      doGet(request, response);
  }
}
```

Hello.htm

```
<html>
  <body>
    <form action = "HelloForm" method = "POST">
      First Name: <input type = "text" name = "first_name">
      <br />
      Last Name: <input type = "text" name = "last_name" />
      <input type = "submit" value = "Submit" />
    </form>
  </body>
</html>
```